

A Study of Kernel Telemetry Options for Security-Oriented Provenance

Paul R. B. Houssel^{1,2}[0009-0009-8302-4393], Olivier
Levillain²[0000-0002-0558-5015], Sylvie Laniepce¹, Nicolas
Dejon¹[0000-0002-2802-0922], and Hervé Debar²[0000-0002-1344-4167]

¹ Orange Research

s.laniepce@orange.com

nicolas.dejon@orange.com

² Samovar, Télécom SudParis, Institut Polytechnique de Paris, France

first.last@telecom-sudparis.eu

Abstract. Provenance aims to capture the origins, transformations, and interactions of system objects for security and forensic applications. Existing provenance capture approaches still face major challenges and are not yet ready for production environments. In this paper, we first analyze the main kernel telemetry capture approaches, identifying eBPF as the most promising, and complement this analysis with micro benchmarks to assess its performance overhead and the filtering mechanisms used to achieve capture granularity, such as restricting capture to individual containers. Building on this foundation, we then classify, according to the studied capture approaches and filtering methods, eight *provenance systems* and five *capture agents* that could serve as their capture layers, collectively referred to as *tools*. Our study reveals that these *tools* are built on highly heterogeneous capture layers, most of which cannot guarantee the integrity and availability of the captured events, completely failing to meet the requirements of security-oriented use cases.

Keywords: Provenance · Kernel · Linux · eBPF

1 Introduction

Provenance in computing systems involves recording, analyzing, and leveraging information about the origins, transformations, and interactions of kernel objects, which are accessed and manipulated by user applications through system calls [4]. For security-oriented provenance, this information is typically represented as a provenance graph that captures the allocation, modification, and use of these objects, together with the subject responsible for those actions. This historical record is essential for security tasks such as intrusion detection through classification of graph embeddings (e.g., generated by a Graph Neural Network) [23], attack signature identification using graph queries [3], post-mortem forensic auditing [32], and active threat hunting [26]. Compared with

identifying malicious signatures in linear audit traces, provenance graphs preserve causal relationships across time and objects, enabling analysts to express and investigate more complex attack behaviors.

Provenance systems are designed for whole-host environments, capturing system events at the kernel level to monitor deployed production systems [33]. These approaches still face several challenges [20] across their three main architectural layers. At the capture layer, which collects kernel-object allocations and events, these include performance overhead, capture integrity and availability, and the difficulty of determining which kernel objects and system events must be captured to support reliable security analysis. In the reduction layer, incoming telemetry is filtered to achieve a desired level of granularity, such as limiting capture to a specific process or a set of files. At the infrastructure layer, where audit traces are processed and managed, provenance graphs can grow significantly in size, complicating storage and downstream security analysis. These challenges are further amplified in cloud-native environments [22], where containers share a single kernel and the capture layer must distinguish the events of individual containers from those of the host system to achieve container-level granularity.

In this paper, we first classify the kernel telemetry capture approaches and identify extended Berkeley Packet Filter (eBPF) as the most promising (§ 3), then empirically evaluate its overhead across program types together with the filtering techniques that enable fine-grained capture (§ 4). This qualifies and quantifies how the choice of capture method affects the security applications of provenance graphs, and shows that filtering is essential for production environments, especially in cloud-native computing. We then classify eight *provenance systems* and five *capture agents* that could serve as provenance capture layers, showing that each is built upon the capture approaches and filtering methods studied above, and macro benchmark the open-source subset (§ 5). To our knowledge, this is the first work to systematically study and compare the capture layers across *provenance systems* and their impact on security-oriented provenance.

2 Related work

Prior surveys have explored *provenance systems* from several complementary angles. Perez et al. [33] provide a systematic review of provenance challenges across different system layers and environments, offering a generic analysis of these issues. Inam et al. [20] extend this effort by formalizing the architectural layers of *provenance systems* and structuring the challenges associated with each layer. Lee et al. investigate provenance in distributed environments [25]. Tan et al. [38] survey integrity mechanisms and encryption schemes for protecting provenance logs in practice. Other studies focus on the detection layer, i.e., the use of provenance graphs for threat detection and forensic analysis. Li et al. [27] study how to design efficient investigations over large provenance graphs, Zipperle et al. [48] explore provenance-based intrusion detection systems leveraging machine learning techniques, and Dong et al. [11] provide an industrial perspective by evaluating the usability and operational effectiveness of such approaches.

These works establish the security value of provenance graphs but largely treat the capture layer as a given, assuming that it records events faithfully and completely. Only the *provenance system* eAudit [34] evaluates capture-layer availability, and none assess integrity or completeness. Moreover, although eBPF has become the dominant capture approach and filtering mechanisms are essential in production environments, no prior work empirically compares the performance trade-offs of eBPF program types and filtering strategies for provenance capture. We address these gaps in the following sections.

3 Kernel telemetry capture approaches

Kernel telemetry captures kernel activity, offering host-wide visibility. Although harder to interpret than application-level telemetry, it is application-agnostic, making it more robust and reliable for security use cases. Vulnerabilities confined to user-space memory (e.g., a heap overflow) may remain invisible in the kernel, but the post-exploitation phase rarely does. From the *provenance systems* and *capture agents* surveyed in § 5, we identify different capture approaches (Table 1) and compare them across five security-relevant properties: 1) *portability*: how easily it deploys across kernel versions; 2) *efficiency*: the performance overhead imposed; 3) *visibility*: the coverage of telemetry data; 4) *safety*: the risk of system instability (e.g., kernel panics, memory corruption); and 5) *robustness*: whether it introduces attack vectors affecting trace integrity and availability. Because the *robustness* challenges largely generalize across capture approaches, we examine this property across all of them in detail (§ 3.3). We distinguish kernel telemetry by its capture location: from the user space or kernel space.

Table 1: Capture approaches for kernel telemetry (● good, ◐ moderate, ○ poor).

location	approach	<i>portability</i>	<i>efficiency</i>	<i>visibility</i>	<i>safety</i>	<i>robustness</i>	example
user space	ptrace	◐	○	○	◐	○	ALASTOR [9]
	fs snapshot	◐	●	○	●	○	ARTISAN [45]
kernel space	integrated systems	○	◐	◐	●	◐	CLARION [5]
	out-of-tree kernel module	○	●	●	◐	◐	LTTng [29]
	eBPF	◐	●	●	●	◐	ProvBPF [28]

3.1 Capture from user space

Kernel telemetry can be collected from user space using two main approaches. The first uses the `ptrace` system call to attach to a process and monitor its system calls and signals. Context switching between user and kernel space renders it *inefficient*, and it lacks *visibility* into kernel internals, threads, and lower-level events such as network packets. In restricted environments without host access,

such as serverless containers, `ptrace` remains the only capture mechanism; e.g., ALASTOR [9] employs STRACE [36] (which relies on `ptrace`). Although long available in Linux, its *portability* is only partial, as the system-call ABI evolves [19]. Prior privilege-escalation vulnerabilities highlight inherent *safety* risks [6].

The second approach infers telemetry from user-space observations; e.g., ARTISAN [45] captures periodic file system snapshots to infer causality from the differences. By avoiding continuous interception, it is more *efficient* than `ptrace`-based methods and introduces no *safety* issues. However, its *visibility* is limited to file-system state changes, omitting other kernel events and objects (e.g., process and network activity); dependence on specific OS or kernel versions limits *portability*; and *robustness* is even weaker, as an attacker can modify the file system between snapshots, leaving no trace in the inferred telemetry.

3.2 Capture from kernel space

Kernel telemetry can also be collected directly from within the kernel, through integrated systems, out-of-tree kernel modules, or eBPF programs. Operating in the kernel generally provides *efficiency* and deep *visibility* into kernel telemetry.

Integrated capture systems FTRACE and AUDITD are *tools* built into the kernel source. AUDITD serves as the capture layer for *provenance systems* such as WINNOWER [18], CLARION [5], and TRACE [21]. Despite adoption, these approaches fall short for three reasons: 1) kernel source modifications require lengthy peer-review, ensuring *safety* but delaying updates and hindering *portability*; 2) this *portability* limit forces adaptations into user space, restricting fine-grained capture to the reduction layer, which we later show is less efficient than filtering in the capture layer; and 3) originally built for debugging or targeted audits rather than continuous whole-system capture, they are less *efficient* under the sustained event rates of long-running, system-wide security monitoring [34].

The kernel also hosts Linux Security Modules (LSMs) e.g., SELinux [35], which enforce Mandatory Access Control (MAC) via dedicated *LSM* interfaces. While LSMs are typically in-tree modules shipped within the kernel source, they can also be distributed as out-of-tree kernel modules or eBPF programs.

Out-of-tree kernel modules Out-of-tree kernel modules are compiled outside the main kernel source tree and mapped into the kernel’s address space, extending functionality without modifying the source code. While most act as drivers, some implement security features [1]. They are the least *portable* approach considered: a module’s implementation is tightly bound to a single kernel version, depending on internal kernel APIs and data structures that change across releases [19], so every target kernel requires a separately adapted build. *Safety* concerns also arise: unlike in-tree modules, they bypass the kernel’s peer-review process, and a poorly implemented module can cause kernel panics or memory corruption (e.g., out-of-bounds writes).

eBPF programs eBPF programs are a *safer* alternative as they extend the kernel through a non-Turing-complete language that guarantees termination (bounded loops, limited instruction counts) [42], while typing and fixed-size allocations enforce memory safety. eBPF generalizes the classic Berkeley Packet Filter from network packet inspection to arbitrary kernel instrumentation. It is partly *portable*: available since kernel v3.18, though the relevant program types arrived later (from v4.1; Table 2) and their attach interfaces may change across versions, constraining version-agnostic deployment [19,47]. We identify four major program types for provenance capture (*LSM*, *tracepoint*, *kprobe* and *tracing*), each attaching to specific kernel interfaces.

LSM programs attach to *LSM* interfaces, historically used by LSMs such as SELinux to enforce MAC policies, where multiple LSMs can be chained, each invoked sequentially to allow or deny the operation. eBPF *LSM* programs use the *LSM_MAC* attach type (all processes) or *LSM_CGROUP* (a given *cgroup*), dispatched through a BPF *trampoline*, a stub the kernel patches in at attach time to jump directly into the program. These interfaces offer more stable ABIs than kernel functions [19], but a single MAC operation may map to multiple system calls, preventing system-call granularity.

tracepoint programs attach to predefined, stable *tracepoint* interfaces that cover system call entry and exit points as well as other kernel semantic events, via a static *NOP* patched into a call upon attachment. The patched call does not reach the eBPF program directly: it enters the kernel’s generic tracepoint dispatcher, which on each firing copies the site’s fixed arguments into a record and passes it to every registered handler.

kprobe programs attach to nearly any kernel function entrance (*kprobe*) or exit (*kretprobe*), as well as to user-space function entry (*uprobe*) and exit (*uretprobe*). The target instruction is replaced with an interrupt (e.g., *INT3* on x86), which triggers the eBPF program before resuming execution. While offering extensive *visibility*, including over *LSM* hooks (functions with the “*security.*” prefix), they incur additional overhead from CPU interruptions and attach to less stable interfaces that may change across versions.

tracing programs cover a similar scope to *kprobe* (excluding user-space functions) but, like *LSM* programs, attach through a BPF trampoline rather than an interrupt: with *CONFIG_FUNCTION_TRACER*, attachment turns the entry *NOP* into a trampoline call that runs the *fentry/fexit* programs around the function, avoiding *kprobe*’s CPU interruptions. Like *tracepoint*, it starts from a patched *NOP*, but the target differs: instead of the generic dispatcher with its per-firing argument copying, the trampoline is generated at attach time for that specific program and function, handing the function’s register-held arguments straight to the program and leaving minimal per-event work.

Table 2 summarizes eBPF program types. Among them, *LSM* programs stand out as the most suitable choice for kernel-telemetry provenance: they enforce policies at a granular level, particularly within containers through *cgroup* attachments, and are more stable than *kprobe* and *tracing* programs, as they adapt to internal kernel changes [19]. Their scope, however, is limited to MAC

operations, raising the question of whether this is sufficient to construct complete provenance graphs. The Linux kernel v6.0 requirement for *cgroup*-attached *LSM* programs is not a significant barrier: among kernels older than 6.0, only 5.15 and 5.10 still receive long-term support, and both reach end of life in December 2026 [40].

Table 2: Comparison of eBPF program types used for provenance capture layers.

Program type	Attach type	TOCTOU resistance	Granularity				Stability	Kernel version ≥	
			system calls	kernel function	user-space function	MAC operation			cgroup
LSM	LSM_MAC	●	○	○	○	●	○	●	5.7
	LSM_CGROUP	●	○	○	○	●	●	●	6.0
tracepoint	tracepoint	▶	●	▶	○	○	○	●	4.7
kprobe	kprobe/kretprobe	▶	●	●	○	●	○	○	4.1
	uprobe/uretprobe	○	○	○	●	○	○	○	4.3
tracing	fentry/fexit	▶	●	●	○	●	○	○	5.5

3.3 A closer look at the *robustness* property

Among user-space methods, *robustness* is weak. With *ptrace*-based tracing, a malicious process can occupy its single tracer slot (e.g., with `PTRACE_TRACEME`), blocking other tracers. With snapshot-based approaches, an attacker may execute critical steps between two snapshots, leaving no evidence in the inferred telemetry. Furthermore, user-space methods can be exposed to unprivileged processes [6], whereas kernel-based approaches run behind strict privilege boundaries and are therefore more *robust*. Kernel modules and eBPF programs further strengthen *robustness*, as by default only privileged users can load them.

However, all approaches that trace system calls are vulnerable to Time-Of-Check to Time-Of-Use (TOCTOU) race-condition attacks [17]: an adversary can alter system call arguments after the check but before kernel processing, compromising capture integrity (e.g., opening a suspicious file while a benign filename is traced). The race window can be widened, for instance via a custom page-fault handler (`userfaultfd`) or a delayed TCP handshake on a controlled server. This affects any mechanism hooking system call entry points that accept user-space pointers. In Linux 6.14 (Table 3), 69% of system calls take at least one such pointer and are therefore vulnerable; among them, 96 point to a filename or file handle and 8 to a socket address, enough to conceal major attack steps. The rest are mostly buffers or call-specific option structures and flags, which security telemetry rarely captures, further limiting visibility. The issue is even more pronounced in security monitoring *tools*: over 70% of system calls in Falco and Tracee attack signatures accept a user-space pointer (Appendix A).

eBPF *LSM* programs distinguish themselves here: security checks occur after arguments have been copied into kernel space, at deeper points along the execution path, conferring stronger resistance to TOCTOU races [17,43]. The three other eBPF program types remain vulnerable when attached to system calls.

Table 3: Prevalence of system calls with user-space pointer arguments.

Set of system calls	# system calls	# of those with a pointer that is	
	with ≥ 1 user-space pointer	file-related	socket-address
All system calls (Linux v6.14)	282/407 (69%)	96	8
$\in$ Falco attack signatures [12]	30/41 (73%)	20	5
$\in$ Tracee attack signatures [41]	14/19 (74%)	9	1

4 Empirical evaluation of eBPF capture

Based on our evaluation of capture approaches (Table 1), eBPF stands out as the most promising for security-oriented provenance: it is the only approach that fully satisfies *efficiency*, *visibility*, and *safety*, while its *robustness* and *portability* depend on which kernel interfaces are instrumented. We therefore empirically evaluate its overhead across four eBPF program types: *tracepoint*, *kprobe*, *tracing* and *LSM* (§ 4.2) and across the filtering methods that achieve capture granularity, an important and understudied aspect for production environments (§ 4.3).³

4.1 Experimental setup

All experiments are conducted on a 24-core host running Debian 12 (Linux 6.1.0-amd64). We test each capture configuration with three benchmarks representing common production workloads: 1) *network* workload, using *httperf* [30] to generate 50k HTTP connections from a client directly connected to a local python HTTP server; 2) *file* workload, using *postmark* [24] to write 50k files of 512 KB each, 500 accessed simultaneously; and 3) *process* workload, using *shbm* [34] to execute the `echo` binary sequentially 50k times. The traced services run inside Docker containers. We quantify performance overhead as the relative CPU-time increase between a traced workload and an untraced baseline, averaged over 10 runs after 10 warm-up iterations, with fixed P-states and disabled C-states. All programs share the same eBPF maps, layouts, and user-space logger, keeping memory overhead constant, and each records the PID, UID, TGID, executable name, and *cgroup* ID of the triggering task, ensuring semantic equivalence.

³ All evaluations can be reproduced: <https://doi.org/10.6084/m9.figshare.33415552>

4.2 Overhead by eBPF program type

Because the program types cannot all attach to the same hooks (*LSM* programs cannot attach to system calls, and *tracepoint* programs cannot attach to *LSM* hooks), we compare them at two locations. At the *system call location*, we instrument *tracing*, *kprobe*, and *tracepoint* programs; at the *LSM location* we attach *LSM* programs to the *LSM* interfaces and *kprobe* and *tracing* programs to the kernel functions that invoke them (prefixed with `security_`). Some programs embed filters so that all capture exactly the same semantic event (connection acceptance for *network*, file creation for *file*, and task cloning for *process*) and our eBPF *LSM* program is registered last in the *LSM* chain so that it only captures operations allowed by all preceding LSMs. For every program type but *LSM*, capturing these events requires a dual attachment, at entry and exit, to respectively read the arguments and discard rejected operations: system calls rejected by the kernel or by an LSM enforcing MAC [8], and, at the *LSM location*, operations denied by another LSM such as SELinux after the `security_` wrapper has been entered. We account for this added cost in our measurements. Table 4 summarizes the kernel interfaces instrumented in this experimental setup.

Table 4: Kernel interfaces instrumented in our experimental setup.

Tracing location	Traced interface per workload (captured event)			eBPF program type
	<i>network</i> (accept conn.)	<i>file</i> (file creation)	<i>process</i> (clone)	
<i>system call</i>	<code>accept</code> , <code>accept4</code>	<code>creat</code> , <code>open</code> ^a , <code>openat</code> ^a , <code>openat2</code> ^a	<code>clone</code> , <code>clone3</code>	<i>tracepoint</i> , <i>kprobe</i> , <i>tracing</i>
<i>LSM</i>	<code>socket_accept</code>	<code>inode_init_security</code>	<code>task_alloc</code> ^b	<i>LSM</i> <i>kprobe</i> , <i>tracing</i>

^a `O_CREAT` flag filtered to capture only file creations. ^b `PF_KTHREAD` flag filtered to sieve kernel threads.

At the *system call location* with entry-only attachment, *tracing* is the most efficient across all workloads: 0.76% (*network*), 8.12% (*file*), and 1.15% (*process*), against 1.43%, 8.63%, and 1.22% for *tracepoint* and 1.66%, 9.10%, and 1.30% for *kprobe* programs. This ordering follows the attachment mechanism: *tracing* and *tracepoint* rely on NOP patching rather than *kprobe*’s CPU interruptions, *tracing* stays ahead because its trampoline calls the program directly from the function prologue, whereas *tracepoint*’s generic dispatcher copies a fixed argument record at every firing, an extra per-event cost.

Fig. 1 reports these results. The solid (*system call location*) and hatched (*LSM location*) bars give the cost of the entry attachment, which captures the arguments; the dotted segments give the additional cost of the exit attachment, needed to observe the verdict and discard rejected events. Only *LSM* programs obtain both with a single attachment: registered last in the chain, they execute only for operations allowed by all other LSMs.

At the *LSM location* with entry-only attachment, *tracing* remains the most efficient across all workloads: 0.82% (*network*), 7.01% (*file*), and 1.04% (*process*), against 1.10%, 8.99%, and 1.10% for *kprobe* and 0.89%, 7.27%, and 1.13% for *LSM* programs, although the gap with *LSM* is negligible given the variance and explained by their attachments: *tracing* programs attach a trampoline directly to the `security_` function prologue, whereas *LSM* programs, attached through the same mechanism, are additionally dispatched through the LSM framework. This advantage does not survive verdict capture: also tracing the `security_` wrapper’s return raises *tracing* to 1.22%, 10.01%, and 2.04% and *kprobe* to 1.40%, 12.99%, and 2.10%, making *LSM* programs the cheapest at this location across all workloads. The *LSM location* is already cheaper than the *system call location* even before counting the second attachment, most markedly for the *file* workload: the *LSM* interface `inode_init_security` fires only on file creation (50k executions), whereas system-call programs must inspect every `open` (100k, as each file is also reopened for reading) for the `O_CREAT` flag to isolate the 50k creations, a count doubled to 200k by the entry/exit dual attachment.

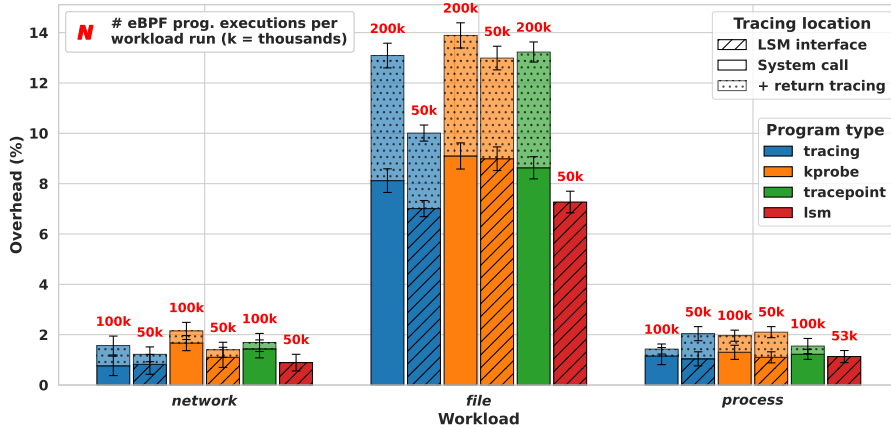


Fig. 1: Performance overhead among eBPF programs.

For provenance, this second attachment is not optional: an operation recorded at entry may still be denied by the kernel or by an LSM such as SELinux. These blocked attempts are valuable to observability, revealing what an attacker tried, but a provenance graph must contain only the causal events that actually occurred. In sum, once the verdict is captured (dotted segments), *tracing*, *kprobe*, and *tracepoint* programs roughly double their per-event work [8], and *LSM* programs, despite *tracing*’s lower per-hook cost, are the most efficient for provenance capture, obtaining the same semantic event (arguments and verdict) with a single attachment.

Overall, performance overhead is highest in the *file* workload, significantly exceeding that of the *network* and *process* workloads. This is due not only to the heavier argument parsing but also to the inherent latency of kernel operations like `inode_init_security`, which involve the label recomputation and initialization of the new inode [44,46].

4.3 Overhead by filtering granularity

In this section, we study container granularity. While our scope is not limited to cloud-native computing, containers are the prevalent deployment unit in production and make the need for granularity concrete: the capture layer must separate a monitored workload’s events from those of the host and co-located workloads, e.g., to trace a single exposed service on a multi-tenant host. The filtering methods we compare apply unchanged to other granularities.

Provenance capture can operate at different granularities, targeting events (e.g., specific system calls), objects (e.g., specific processes or users), or containers. Such granularity is essential in production: capturing all kernel activity inflates overhead and data volume, whereas filtering at the capture layer retains only the subset relevant to the security analysis. eBPF programs achieve event granularity by attaching to specific interfaces, such as system calls, kernel or user-space functions, and to MAC operations (Table 2), and object granularity by filtering event attributes within the program itself. *LSM* programs can additionally achieve container granularity by attaching to a specific *cgroup* (unlike other eBPF program types): every container is assigned a dedicated *cgroup* by the OCI-standard runtimes (e.g., `runc`) underlying Docker, Podman, containerd, and CRI-O. Its file descriptor, located in the *cgroup* file system (`/sys/fs/cgroup`), can be used to create an eBPF link [13] that attaches an eBPF program to the *cgroup*, so that it triggers only for events from processes within that *cgroup*.

Cgroup attachment filters events before the eBPF program is triggered, referred to as *pre* filtering [7]. The other program types reach the same granularity by filtering within the program (*in*), using the `bpf_get_current_cgroup_id` helper to compare the event’s *cgroup* ID with a target ID; the filtering instructions then execute for every event, which may increase overhead. Filtering can also occur after the program executes (*post*), collecting all events and deferring filtering to user space, serving container-granular data at query time. These methods generalize to capture approaches beyond eBPF.

We evaluate the overhead of these filtering methods, reusing the setup of § 4.1 with one change: each container is deployed twice. Both instances run the same workload, but only one is traced; the second generates irrelevant events that *in* and *post* filtering must process but *pre* avoids, making the overhead difference observable. Fig. 2 presents the measured overhead and number of written traces per filtering method and program type (*pre* applies only to *LSM*, as seen before).

We observe that *pre* filtering introduces the lowest overhead across all workloads. For example, under *process* workload, the *LSM* program incurs 0.45% overhead with *pre*, compared to 0.85% with *in* and 1.66% with *post* filtering. *In* filtering is more expensive since its filtering logic executes for every event,

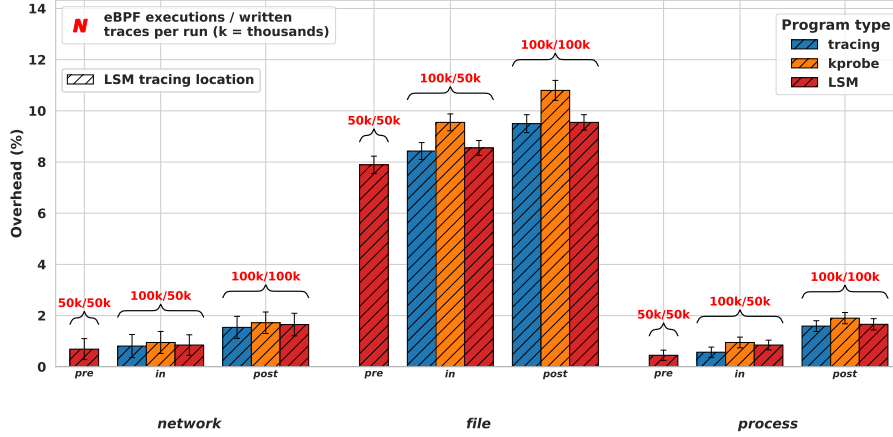


Fig. 2: Performance overhead as a function of the filtering used to achieve container granularity at *LSM* location.

while *post* filtering yields the highest overhead: for the *LSM* program, it nearly doubles the overhead of *in* under *network* workload (1.65% vs. 0.85%) and adds 1 percentage point under *file* workload (9.55% vs. 8.55%).

Notably, *LSM* programs with *pre* filtering not only outperform the other program types with *in* filtering, but also surpass *tracing* programs, the type with the lowest per-hook cost (Fig. 1). Indeed, *pre* filtering achieves overheads of, respectively, 0.69% (*network*), 7.89% (*file*), and 0.45% (*process*) for the *LSM* program, all lower than the corresponding values for the *tracing* program with *in* filtering (resp. 0.81%, 8.43%, and 0.57%). These findings highlight the efficiency advantage of direct *cgroup* attachment: making *LSM* programs even more attractive for *provenance systems* thanks to their *pre* filtering mechanism.

The red annotations report the eBPF executions and written traces per run, which explain these differences: *pre* only executes for relevant events, *in* executes for every event but writes only the relevant traces, and *post* both executes and writes for every event, hence its higher overhead and its doubled trace volume.

5 Classification of capture layers

Building on the capture approaches and eBPF evaluation of the previous sections, we now classify existing *provenance systems* and *capture agents* according to these mechanisms and macro benchmark the open-source subset. We exclude STRACE, ALASTOR, and ARTISAN, as their user-space capture prevents any of them from satisfying all four attributes essential to security-oriented provenance simultaneously: *efficiency*, *visibility*, *safety*, and *robustness* (§ 3).

5.1 Provenance systems

Several contributions have proposed *provenance systems* with distinct capture layers, each addressing a specific limitation. We observe that some capture *LSM* interfaces while others capture system calls.

Among the former: CamFlow [31,32] is deployed as an LSM that leverages the Netfilter and *LSM* interfaces to monitor network connections alongside process and file activities. It supports fine-grained *in* filtering on specific kernel objects or events. To achieve complete coverage of all information flows between kernel objects, CamFlow depends on a patched kernel that adds further *LSM* interfaces; today only a subset of these proposed interfaces is in the Linux kernel. ProvBPF [28] captures at container granularity through saBPF, a patched kernel that adds kernel-object storage and lets *LSM* programs attach directly to specific *cgroups*, avoiding the overhead of *in* or *post* filtering of irrelevant host events. As discussed earlier (§ 4.3), a similar capability has since been adopted in the Linux kernel 6.0 [13]. ConProv [10] focuses on accurately representing container activity, addressing limitations such as unreliable absolute file-path resolution within containers and the lack of security context (e.g., used capabilities). It uses *tracepoint* programs to trace system calls and *kprobe* programs to trace *LSM* operations (e.g., `security_capable`) tied to capability checks.

Among the systems capturing system calls: SPADE [14], which relies on AUDITD, was one of the first to model captured system calls as provenance graphs and to implement a complete capture-to-analysis pipeline. WINNOWER, TRACE, and CLARION are all built on the SPADE framework, though their source code has never been released. WINNOWER proposes an algorithm to condense graph size without losing security-relevant semantics. TRACE aggregates provenance capture across multiple network nodes. CLARION targets container provenance, distinguishing mount namespaces for path names and modeling container boundaries in the graph, but does not represent orchestration-specific semantics. All of these *tools* rely on AUDITD, which lacks efficiency and robustness and offers only limited visibility for security provenance (§ 3.2 & 3.3). eAudit [34] instead uses eBPF *tracepoint* programs for 81 system calls (a superset of those monitored by TRACE), mitigating data loss, overhead, and tampering through a compact encoding scheme, per-CPU ring buffers, and system-call prioritization.

Beyond this survey, we empirically evaluate the open-source *provenance systems*, reusing the methodology of § 4.1. Because some require a patched kernel, we deploy each on a distinct Fedora 35 server VM, the only distribution targeted by their patched kernels and build environments, which makes every tool readily reproducible. On each VM, we run the benchmark with and without capture, keeping each system’s defaults. Fig. 3 reports, for every evaluated *tool*, the performance overhead, captured provenance graph size, estimated log loss, and monitored kernel interfaces. We quantify log loss as $(1 - n_{\text{traced}}/n_{\text{expected}}) \times 100$ over a per-workload *witness event* whose ground-truth count is known (e.g., 50k `clone` events for the *process* workload).

Fig. 3 shows that the evaluated systems differ widely along two axes: the performance overhead they impose and the fraction of logs they actually capture.

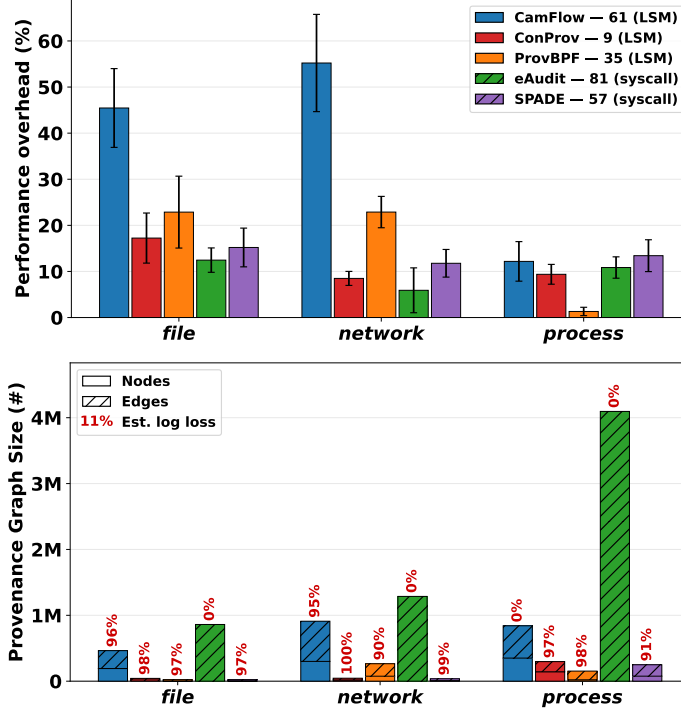


Fig. 3: Macro benchmark of open-source *provenance systems*.

Considering the overhead in isolation is misleading, since most systems lose a considerable share of their logs, and each one instruments a very different set of kernel interfaces (e.g., ConProv monitors only 9 kernel interfaces).

eAudit stands out: despite instrumenting the most interfaces (81 system calls), it incurs among the lowest overheads (5.9% on *network*, 12% on *file*) and loses no logs in any workload. This is because it does not reconstruct a provenance graph but emits a raw stream of system calls and their arguments; combined with per-CPU ring buffers, this keeps it efficient. Consequently, its output is an edge stream (e.g., 4.1M edges in the *process* workload) rather than a node/edge graph, so it never materializes the large graphs the modeling-based *tools* would have produced had they not dropped logs.

The four *tools* that build provenance graphs (CamFlow, ProvBPF, ConProv, and SPADE) all suffer substantial log loss (90 to 100% in most workloads), thus producing far smaller graphs. ConProv is the most extreme case: it instruments only 9 interfaces and loses more than 97% of logs. SPADE, built on AUDITD, combines the highest *process* overhead (13.4%) with 91–99% loss. CamFlow incurs the highest overhead of any evaluated system (45% on *file*, 55% on *network*) and still loses 95–96% of logs in two workloads. It produces the largest graphs and

is the only graph-modeling *tool* to achieve zero log loss in the *process* workload (351k nodes / 494k edges). Even with high loss, CamFlow produces large graphs due to its modeling choices: it represents files, inodes, and paths as distinct graph nodes, so even a 96% log loss yields a big *file* graph (192k nodes / 272k edges); similarly, modeling every network packet as its own node produces the largest *network* graph (300k nodes / 610k edges) despite roughly 95% of logs being lost.

Overall, no evaluated *provenance system* reconciles overhead, log loss, and actually modeling the graph: the only *system* that avoids log loss and is efficient (i.e., eAudit) omits graph construction.

5.2 Capture agents

Beyond the *provenance systems* above, several widely adopted *capture agents* collect kernel telemetry for broader observability. Unlike *provenance systems*, which build provenance graphs over long time windows, these agents monitor for limited windows (e.g., debugging or incident response) and can thus afford to trace broader event sets, up to every system call. Serving a different purpose, they cannot be directly compared to *provenance systems*; we therefore place their macro benchmark in Appendix B. In sum, monitoring a large, system-call-heavy interface set yields large log volumes and considerable log loss.

LTTng [29] is an out-of-tree kernel module capturing system calls and kernel events via *tracepoints*, kernel functions via *kprobes*, and user-space telemetry from Python, Java, and C/C++; unlike eBPF, such modules offer weaker safety, an acceptable trade-off given LTTng’s maturity. Sysdig, Tracee, and Tetragon are eBPF-based agents for granular observability and rule-based detection. Sysdig traces system calls with *tracing* programs and other kernel events (e.g., page faults) with *tracing* and *tracepoint*, for debugging and security monitoring. Tracee ships predefined *LSM*, *kprobe*, *tracepoint*, and *tracing* programs for specific security, network, and system events. Tetragon targets SOC teams and administrators with an operator-friendly YAML DSL specifying and enforcing capture policies, compiled into *LSM*, *kprobe*, *tracepoint*, and *uprobe* programs.

Their strengths for provenance are 1) a mature capture model with predefined programs identifying high-level system events (e.g., container creation or kernel module loading), and 2) strong industrial support ensuring portability.

5.3 Classification

To compare these *provenance systems* and *capture agents*, we classify them in Tables 5 and 6 along five key attributes of their capture layer: capture approach, cloud-native awareness, capture granularity, open-source availability, and maintenance. We discuss each one by drawing on the previous empirical evaluations.

Capture approach (Column 2): mechanism to capture kernel telemetry.

Capture mostly occurs in the kernel via eBPF, dominant among the most recent contributions, in line with the benefits analyzed in § 3.2. Several *provenance*

Table 5: *Provenance systems.*

Provenance system	Capture approach	Cloud-native awareness		Capture granularity			Open source availability	Maintained
		container orchestration		<i>pre</i>	<i>in</i>	<i>post</i>		
SPADE [14] (2012)	AUDITD	○	○	system call	∅	event attribute	●	●
CamFlow [31] (2017)	LSM (patched <i>LSM</i> , Netfilter)	○	○	MAC operation network connection	kernel object kernel event	∅	●	○
WINNOWER [18] (2018)	AUDITD	●	○	system call	∅	∅	○	○
TRACE [21] (2021)	AUDITD	○	○	system call	∅	∅	○	○
ProvBPF [28] (2021)	eBPF (patched <i>LSM</i>)	●	○	MAC operation container	∅	∅	●	○
CLARION [5] (2021)	AUDITD	●	○	system call	container	∅	○	○
ConProv [10] (2024)	eBPF (<i>kprobe</i> , <i>tracepoint</i>)	●	○	system call capabilities socket connection file permission	container	∅	●	○
eAudit [34] (2024)	eBPF (<i>tracepoint</i>)	○	○	system call	∅	∅	●	○

Table 6: *Capture agents* that could serve the capture layer of *provenance systems*.

Capture agent	Capture approach	Cloud-native awareness		Capture granularity			Open source availability	Maintained
		container orchestration		<i>pre</i>	<i>in</i>	<i>post</i>		
AUDITD [2] (2004)	kernel audit subsystem (<i>syscall</i> , <i>LSM</i>)	○	○	system call MAC operation file watch	∅	event attribute	●	●
LTTng [29] (2005)	out-of-tree kernel module (<i>kprobe</i> , <i>tracepoint</i> , programming libraries)	●	○	system call kernel function kernel event C/C++, Java & Python program	∅	event attribute (incl. container)	●	●
Sysdig [37] (2014)	eBPF (<i>tracing</i> , <i>tracepoint</i>)	●	○	system call kernel event	∅	event attribute (incl. container)	●	●
Tracee [41] (2020)	eBPF (<i>LSM</i> , <i>tracing</i> , <i>kprobe</i> , <i>tracepoint</i>)	●	●	MAC operation system call kernel event network event	process, binary namespace container, pod <i>cgroup</i> , user	∅	●	●
Tetragon [39] (2022)	eBPF (<i>LSM</i> , <i>kprobe</i> , <i>tracepoint</i> , <i>uprobe</i>)	●	●	MAC operation system call kernel function user-space function	func. argument process, binary namespace container	event attribute	●	●

systems, however, still build on AUDITD (i.e., SPADE, WINNOWER, TRACE, and CLARION among those surveyed), despite the limitations discussed above (§ 3.2 & 3.3). Within these eBPF-based *tools*, capture mostly relies on the *LSM*, *tracepoint*, or *kprobe* program types (§ 3.2), with *provenance systems* attaching to both entry and exit points of system calls and sometimes adding *kprobe* interfaces for internal kernel functions. The more recent *tracing* program type remains used almost exclusively by *capture agents*, even though its trampoline-based attachment offers the lowest per-attachment overhead we measured (§ 4.2).

Beyond the capture approach, the set of instrumented interfaces differs by *tool* class: *provenance systems* select tailored subsets, whereas *capture agents* instrument broader sets for visibility. Effective provenance, however, requires capturing not only system calls but also kernel-object life cycles (e.g., inode allocation) and information flows between objects [31]. *LSM* interfaces naturally cover these life cycles and most information flows, but their completeness remains an open question: Georget et al. [15,16] used compile-time control-flow graphs to verify which system-call paths reach *LSM* interfaces and, on Linux 4.3, found missing interfaces for pipe, splice, repeated socket messages, memory unmapping, and shared-memory detachments, a subset of which are now in the kernel.

In sum, we observe that *LSM* interfaces are the most attractive basis for capture: they offer low overhead (§ 4 & 4.3), expose kernel-object insights, can be extended for missing information flows, and their smaller surface (265 *LSM* vs. 407 system calls in Linux 6.14, counting every interface regardless of configuration or architecture) yields more abstract graphs with fewer events while preserving semantic richness. System-call-based systems, by contrast, diverge widely in the set they capture, and none assess whether their chosen subset suffices for complete provenance graphs.

Cloud-native awareness (Column 3) assesses whether a *provenance system* or *capture agent* records container- and orchestration-specific events. Container-aware *tools* should trace container runtime entities and their attributes (e.g., IDs, labels, resource constraints, life-cycle states), while orchestration-aware *tools* should additionally capture orchestration-specific events such as pod life-cycles.

Capture agents are generally container-aware, the result of the significant engineering effort behind these large, industry-backed open-source projects; AUDITD is the exception, a long-standing kernel audit facility that predates containers and records neither container nor orchestration semantics. *Provenance systems* lag behind. As Deng et al. [10] note, many of them, such as ProvBPF, identify containers as entities but lack complete records of container-specific security metadata, such as the use of Linux capabilities. Container-awareness is nonetheless attainable with AUDITD: WINNOWER and CLARION group audit events per container and are able to distinguish them from the host. Orchestration-awareness remains exclusive to Tracee and Tetragon.

Capture granularity (Column 4) evaluates how far a system filters events to capture only relevant subsets of provenance data. Granularity can apply to ob-

jects (e.g., processes, users, or containers), events (e.g., system calls or scheduler activity), or event attributes (e.g., timestamps), filtered before capture (*pre*), during instrumentation (*in*), or after capture (*post*); $\emptyset$ indicates no filtering.

Event granularity is mostly achieved through *pre* filtering, by attaching capture only to the relevant interfaces. Container granularity, by contrast, mostly relies on the costlier *in* or *post* filtering; the notable exception is ProvBPF, whose patched saBPF kernel attaches eBPF programs directly to a *cgroup*, reaching container granularity through *pre* filtering.

Open source availability & maintenance (Columns 5 & 6) examine whether each *tool*’s source code is public and if it is actively maintained after publication.

In contrast to the *capture agents*, some *provenance systems* are not open source, and all except SPADE are unmaintained. Even among the open-source ones, portability is limited: CamFlow and ProvBPF depend on custom kernel patches, restricting their deployment to a specific kernel compiled with specific modules and configurations. *Capture agents*, by contrast, are actively developed and frequently commercially backed, offering the stability and long-term usability that make them more suitable foundations for *provenance systems*.

6 Conclusion

We studied the kernel telemetry options for building the capture layer of security-oriented *provenance systems*, working from the capture mechanisms up to the *tools* that build on them: we classified five capture approaches, empirically benchmarked the eBPF program types and the filtering methods that achieve capture granularity, and classified eight *provenance systems* and five *capture agents*, running our macro benchmark on those available as open source.

On *how* to capture, eBPF is now the de facto mechanism, but its program types differ in cost, and *LSM* programs emerge as the best suited for provenance: they capture a MAC operation’s arguments and verdict with a single attachment, where every other program type must hook both entry and exit, whether of the system call or of the `security_` wrapper, expose stable interfaces and reach *pre* filtering through direct *cgroup* attachment. Capturing the verdict is not optional for the use case: an operation recorded at entry may still be denied by an LSM such as SELinux, and while these blocked attempts inform observability, a provenance graph must record only the causal events that actually occurred.

On *what* to capture and on *robustness*, the most overlooked dimension, *tools* fall short. They instrument broader interface sets than the use case requires, yet none verify that their subset captures provenance completely; *LSM* interfaces are nonetheless an attractive basis, exposing kernel-object life cycles over a smaller surface. On robustness, neither integrity nor availability is guaranteed: system-call-based *tools* remain open to TOCTOU evasion, and every graph-building *tool* drops most of its logs in our macro benchmark. *LSM* programs, which inspect arguments already copied into the kernel, offer the stronger integrity guarantees regarding TOCTOU resistance.

Overall, *provenance systems* rely on widely different capture layers, most unable to guarantee event integrity and availability, leaving them unsuitable for security use cases; *capture agents*, by contrast, offer the maintenance and portability that *provenance systems* lack, making them attractive building blocks for future systems, though most remain slow and lossy. A fair evaluation must ultimately look beyond capture to each *tool*'s configurability and the expressiveness of its provenance output: future work should assess the semantic quality of the resulting graphs for downstream security tasks through metrics such as intrusion-detection accuracy and forensic completeness.

References

1. Angelakopoulos, I., Stringhini, G., Egele, M.: FirmSolo: enabling dynamic analysis of binary Linux-based IoT kernel modules. In: Proceedings of the 32nd USENIX Conference on Security Symposium. pp. 5021–5038. SEC '23, USENIX (2023)
2. auditd - The Linux Audit daemon (2004)
3. Blair, W., Araujo, F., Taylor, T., Jang, J.: Automated Synthesis of Effect Graph Policies for Microservice-Aware Stateful System Call Specialization. In: 2024 IEEE Symposium on Security and Privacy (SP). pp. 4554–4572. IEEE (May 2024)
4. Carata, L., Akoush, S., Balakrishnan, N., Bytheway, T., Sohan, R., Seltzer, M., Hopper, A.: A primer on provenance. Communications of the ACM (May 2014)
5. Chen, X., Chen, Y.: CLARION: Sound and Clear Provenance Tracking for Microservice Deployments. In: Proceedings of the 30th USENIX Security Symposium (SEC). pp. 3989–4006. USENIX Association (2021)
6. Connor, E., McDaniel, T., Smith, J.M., Schuchard, M.: PKU Pitfalls: Attacks on PKU-based Memory Isolation Systems. In: Proceedings of the 29th USENIX Security Symposium (SEC). pp. 1409–1426. ACM (2020)
7. Craun, M., Hussain, K., Gautam, U., Ji, Z., Rao, T., Williams, D.: Eliminating eBPF Tracing Overhead on Untraced Processes. In: Proceedings of the SIGCOMM Workshop on eBPF and Kernel Extensions. pp. 16–22. ACM (Aug 2024)
8. Craun, M., Williams, D.: Pairwise bpf programs should be optimized together. In: Proceedings of the 3rd Workshop on eBPF and Kernel Extensions (2025)
9. Datta, P., Polinsky, I., Inam, M.A., Bates, A., Enck, W.: ALASTOR: Reconstructing the Provenance of Serverless Intrusions. In: Proceedings of the 31st USENIX Security Symposium (SEC). pp. 2443–2460. ACM (2022)
10. Deng, Q., Zhang, Y., Xu, Z., Tan, Q., Zhang, Y.: ConProv: A Container-Aware Provenance System for Attack Investigation. In: Annual Computer Security Applications Conference (ACSAC) 2024 Proceedings. pp. 89–101. IEEE (2024)
11. Dong, F., Li, S., Jiang, P., Li, D., Wang, H., Huang, L., Xiao, X., Chen, J., Luo, X., Guo, Y., Chen, X.: Are we there yet? An Industrial Viewpoint on Provenance-based Endpoint Detection and Response Tools. In: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security. pp. 2396–2410. CCS'23, Association for Computing Machinery (Nov 2023)
12. Falco: open source security tool for containers, kubernetes and cloud (2014)
13. Fomichev, S.: bpf: per-cgroup lsm flavor. <https://github.com/torvalds/linux/commit/69fd337a975c7e690dfe49d9cb4fe5ba1e6db44e> (2022), git commit 69fd337
14. Gehani, A., Tariq, D.: Spade: Support for provenance auditing in distributed environments. In: ACM/IFIP/USENIX International Conference on Distributed Systems Platforms and Open Distributed Processing. pp. 101–120. Springer (2012)

15. Georget, L., Jaume, M., Tronel, F., Piolle, G., Tong, V.V.T.: Verifying the Reliability of Operating System-Level Information Flow Control Systems in Linux. In: 2017 IEEE/ACM 5th International FME Workshop on Formal Methods in Software Engineering (FormaliSE). pp. 10–16. IEEE (May 2017)
16. Georget, L., Tronel, F., Tong, V.V.T.: Kayrebt: An activity diagram extraction and visualization toolset designed for the Linux codebase. In: 2015 IEEE 3rd Working Conference on Software Visualization (VISSOFT). pp. 170–174. IEEE (Sep 2015)
17. Guo, R., Zeng, J.: Trace Me If You can: BypassingLinux Syscall Tracing (2022), DEF CON 30
18. Hassan, W.U., Lemay, M., Aguse, N., Bates, A., Moyer, T.: Towards Scalable Cluster Auditing through Grammatical Inference over Provenance Graphs. In: Proceedings 2018 Network and Distributed System Security Symposium (NDSS). pp. 840–854. Internet Society (2018)
19. Houssel, P.R.B., Laniepece, S., Levillain, O.: Towards Provenance for Cybersecurity in Cloud-Native Production Infrastructure. In: 2025 55th Annual IEEE/IFIP International Conference on Dependable Systems and Networks. DSN 2025, vol. Supplemental Volume (DSN-S), pp. 227–229. IEEE (2025)
20. Inam, M.A., Chen, Y., Goyal, A., Liu, J., Mink, J., Michael, N., Gaur, S., Bates, A., Hassan, W.U.: SoK: History is a Vast Early Warning System: Auditing the Provenance of System Intrusions. In: 2023 IEEE Symposium on Security and Privacy (SP). pp. 2620–2638. IEEE (May 2023)
21. Irshad, H., Ciocarlie, G., Gehani, A., Yegneswaran, V., Lee, K.H., Patel, J., Jha, S., Kwon, Y., Xu, D., Zhang, X.: TRACE: Enterprise-Wide Provenance Tracking for Real-Time APT Detection. IEEE Transactions on Information Forensics and Security **16**, 4363–4376 (2021)
22. Jarkas, O., Ko, R., Dong, N., Mahmud, R.: A Container Security Survey: Exploits, Attacks, and Defenses. ACM Comput. Surv. **57**(7), 170:1–170:36 (Feb 2025)
23. Jiang, B., Bilot, T., Madhoun, N.E., Agha, K.A., Zouaoui, A., Iqbal, S., Han, X., Pasquier, T.: ORTHRUS: Achieving High Quality of Attribution in Provenance-based Intrusion Detection Systems. In: Proceedings of the 34th USENIX Conference on Security Symposium (SEC). pp. 7173–7192. ACM (2025)
24. Katcher, J.: PostMark: A New File System Benchmark. Tech. Rep. TR-3022, Network Appliance (NetApp) (1997)
25. Lee, B., Awad, A., Awad, M.: Towards Secure Provenance in the Cloud: A Survey. In: 2015 IEEE/ACM 8th International Conference on Utility and Cloud Computing (UCC). pp. 577–582. IEEE (Dec 2015)
26. Li, J., Zhang, R., Liu, J., Liu, G.: LogKernel: A Threat Hunting Approach Based on Behaviour Provenance Graph and Graph Kernel Clustering. Security and Communication Networks **2022**(1), 4577141 (2022)
27. Li, Z., Chen, Q.A., Yang, R., Chen, Y., Ruan, W.: Threat detection and investigation with system-level provenance graphs: A survey. Computers & Security **106**, 102282 (Jul 2021), publisher: Elsevier BV
28. Lim, S.Y., Stelea, B., Han, X., Pasquier, T.: Secure Namespaced Kernel Audit for Containers. In: Proceedings of the ACM Symposium on Cloud Computing. pp. 518–532. ACM (Nov 2021)
29. LTTng: an open source tracing framework for Linux (2005)
30. Mosberger, D., Jin, T.: httpperf—a tool for measuring web server performance. SIGMETRICS Perform. Eval. Rev. **26**(3), 31–37 (1998)
31. Pasquier, T., Han, X., Goldstein, M., Moyer, T., Eysers, D., Seltzer, M., Bacon, J.: Practical whole-system provenance capture. In: Proceedings of the 2017 Symposium on Cloud Computing (SoCC). ACM (Sep 2017)

32. Pasquier, T., Han, X., Moyer, T., Bates, A., Hermant, O., Eysers, D., Bacon, J., Seltzer, M.: Runtime Analysis of Whole-System Provenance. In: Proceedings of the 2018 ACM SIGSAC Conference. ACM (Oct 2018)
33. Pérez, B., Rubio, J., Sáenz-Adán, C.: A systematic review of provenance systems. *Knowledge and Information Systems* **57**(3), 495–543 (Dec 2018)
34. Sekar, R., Kimm, H., Aich, R.: eAudit: A Fast, Scalable and Deployable Audit Data Collection System. In: 2024 IEEE Symposium on Security and Privacy (SP) Proceedings. pp. 3571–3589. IEEE (May 2024)
35. Smalley, S., Vance, C., Salamon, W.: Implementing SELinux as a Linux security module. NSA, NAI Labs Report, Fort Meade, Maryland, U.S. (2001)
36. strace - the linux syscall tracer (1991)
37. Sysdig: Linux system exploration and troubleshooting tool with first class support for containers (2014)
38. Tan, Y.S., Ko, R.K., Holmes, G.: Security and Data Accountability in Distributed Systems: A Provenance Survey. In: proceedings of the 2013 IEEE 10th International Conference on High Performance Computing and Communications & 2013 IEEE International Conference on Embedded and Ubiquitous Computing (EUC). pp. 1571–1578. IEEE, Zhangjiajie, Hunan Province, P.R. China (Nov 2013)
39. Tetragon - eBPF-based Security Observability and Runtime Enforcement (2022)
40. The Linux Kernel Organization: Active kernel releases. <https://www.kernel.org/category/releases.html> (2026), accessed: 2026-06-15
41. Tracee: Linux Runtime Security and Forensics using eBPF (2020)
42. Vishwanathan, H., Shachnai, M., Narayana, S., Nagarakatte, S.: Verifying the Verifier: eBPF Range Analysis Verification. In: Enea, C., Lal, A. (eds.) *Computer Aided Verification*. pp. 226–251. Springer Nature Switzerland (2023)
43. Watson, R.N.M.: Exploiting concurrency vulnerabilities in system call wrappers. In: Proceedings of the first USENIX workshop on Offensive Technologies. pp. 1–8. WOOT '07, USENIX Association (Aug 2007)
44. Yamauchi, T., Yamamoto, K.: LSMPMON: Performance Evaluation Mechanism of LSM-based Secure OS. *International Journal of Security and Its Applications* **6**(2), 81–90 (2012)
45. Yu, L., Ye, Y., Zhang, Z., Zhang, X.: Cost-effective Attack Forensics by Recording and Correlating File System Changes. In: Proceedings of the 33rd USENIX Security Symposium (SEC). pp. 1705 – 1722. ACM (2024)
46. Zhang, W., Jaeger, T., Liu, P.: Analyzing the Overhead of Filesystem Protection Using Linux Security Modules. In: Proceedings of the 2021 ACM Asia Conference on Computer and Communications Security. pp. 393 – 406. ACM (Jun 2021)
47. Zhong, S.W., Liu, J., Arpaci-Dusseau, A., Arpaci-Dusseau, R.: Revealing the unstable foundations of ebpf-based kernel extensions. In: Proceedings of the Twentieth European Conference on Computer Systems. pp. 21–41 (2025)
48. Zipperle, M., Gottwalt, F., Chang, E., Dillon, T.: Provenance-based Intrusion Detection Systems: A Survey. *ACM Computing Surveys* **55**(7), 1–36 (Dec 2022)

Appendix A Robustness analysis

Syscalls	User-space pointer(s)
accept, accept4	upeer_sockaddr, upeer_addrlen
bpf	attr
chmod, fchmodat, open, openat	filename
connect	servaddr
creat, mkdir, mkdirat, rmdir, unlink, unlinkat	pathname
dup, dup2, dup3, fchmod, listen, ptrace, setns, setuid, socket, unshare, userfaultfd	$\emptyset$
execve, execveat	filename, argv, envp
finit_module	uargs
init_module	umod, uargs
link, linkat, rename, renameat, renameat2, symlinkat	oldname, newname
openat2	filename, how
recvfrom	ubuf, addr, addr_len
recvmsg, sendmsg	msg
sendto	buff, addr
symlink	old, new

(a) Falco

Syscalls	User-space pointer(s)
connect	servaddr
dup, dup2, dup3, mprotect, ptrace	$\emptyset$
execve, execveat	filename, argv, envp
finit_module	uargs
init_module	umod, uargs
mount	dev_name, dir_name, type, data
open, openat	filename
open_by_handle_at	handle
openat2	filename, how
rename, renameat2	oldname, newname
write	buf
writew	vec

(b) Tracee

Table 7: System calls referenced by the Falco and Tracee attack signatures and their user-space (`__user`) pointer arguments.

Appendix B Macro benchmark of open-source *capture agents*

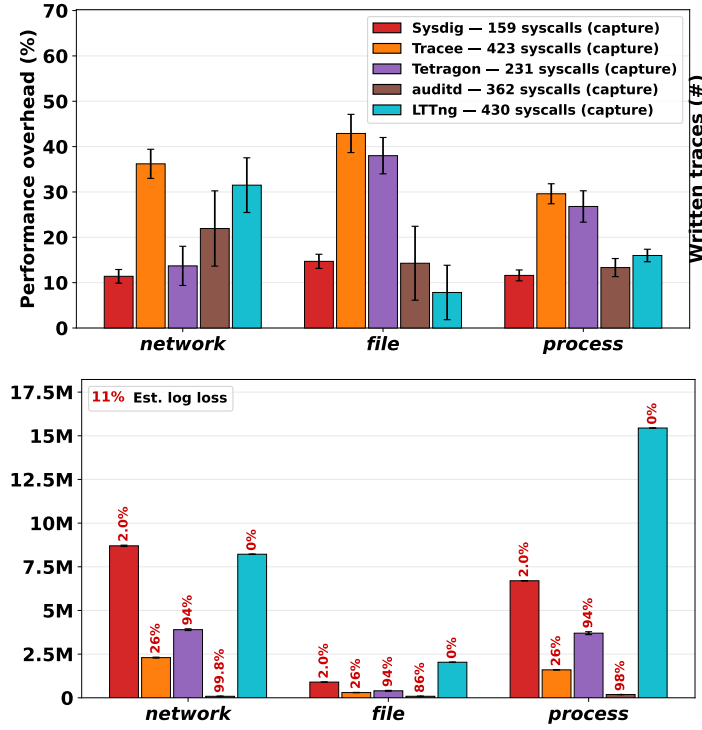


Fig. 4: Macro benchmark of open-source *capture agents*.